

La provenance de la masse grasse

Analyse ciblée d'un seul défaut - Force Tracker ft-v1234, arbre a9de8423 - 23 septembre 2026.

[/!] **Analyse seulement.** Aucun code modifié, aucune correction, aucun bump.

Mesure dans un vrai navigateur sur l'application servie, **profil SYNTHETIQUE etiquete** - aucune donnée personnelle. Ce dossier vit donc dans le depot.

La reponse, en une phrase

US Navy est **correctement étiqueté** comme une estimation au moment où il est enregistré. La provenance se perd **à la relecture** : `leanMassRecente()` ne lit jamais `bfSrc` et écrit nature : 'saisie' en dur. [*] Milo reçoit alors, mot pour mot, « *elle est calculée à partir d'un %% de masse grasse qu'il/elle a SAISI lui/elle-même* » - **faux**, la personne n'a rien tapé. [!!] **Mais l'erreur est PUREMENT SEMANTIQUE** : mesure. L'étiquette ne change **aucun** chiffre.

1. Mécanisme - l'écriture est correcte, et c'est ce qui prouve le diagnostic

_bfNavy() prend **cou, tour de taille, hanches (femme), taille corporelle, sexe**. Dans saveBodyFat il n est appele qu **en repli**, si la case du %% est vide :

L'etiquetage a l'écriture, relu dans le code servi et vérifié par la sonde :

```
tracking.js:994 let bfSrc = (_brutBf && bf) ? BF_MESURE : null; // tape -> 'mesure'
tracking.js:1004 if(!bf){ const navy = _bfNavy(...);
tracking.js:1067 if(navy != null){ bf = navy; bfSrc = BF_ESTIME; } }
if(!garderSrc) e.bfSrc = bfSrc || BF_MESURE; // écrit sur la ligne
MESURE PAR LA SONDE : la ligne enregistrée porte bien bfSrc = 'estime'
```

[ok] **L etiquetage a l ecriture est CORRECT.** Le toast le dit aussi : « *Estimation enregistree* » contre « *Mesure enregistree* ».

2. Point EXACT de perte de provenance

La seule ligne qui compte, relue dans le code servi a chaque generation :

```
state.js function leanMassRecente(){  
    ...  
    const bf = Number(w.bf), bw = Number(w.kg != null ? w.kg : w.bw);  
    if(!(isFinite(bf) && bf>0 && bf<70 && isFinite(bw) && bw>0)) return;  
    cand.push({ date:w.date, lm:..., poids:bw, src:'pesee', nature:'saisie' });  
                                     ^^^^^^^^^^^^^^^^^^^^  
    bfSrc apparait 0 fois dans toute la fonction | nature:'saisie' est ecrit EN DUR (1 fois)
```

Diagnostic : reponse B - et le voisin le prouve

B : US Navy est correctement etiquete, mais `leanMassRecente()` **ignore** `bfSrc`. Ni A, ni C. [*] Le garde central de ce document n est pas un souvenir : la fonction est **extraite de** `state.js` et on verifie qu elle ne mentionne pas `bfSrc`. Si elle se mettait a le lire, ce PDF ne sortirait pas. [*] **Le contraste qui tranche** : `bfDerniere()`, juste a cote dans `tracking.js`, **filtre bien** sur `bfSrc` !**==** `BF_ESTIME`. *Le stockage sait, l affichage sait - seul* `leanMassRecente` *est aveugle*.

3. Les origines de masse grasse reellement supportees par le code

Origine	bfSrc ecrit	Ancre relue
saisie manuelle	'mesure '	tracking.js:994
US Navy (repli)	'estime '	tracking.js:1004
bilan corporel / balance	'mesure '	tracking.js:1990
import de bilans	'mesure '	tracking.js:1388-1389
edition d une pesee	conserve, sinon mesure	tracking.js:1134
ancienne ligne	absent	-

[!/] Le **bilan corporel** n'emprunte pas ce chemin : il alimente S. bodyScans, qui a sa propre branche dans leanMassRecente avec nature: 'lue' / 'deduite' - **correctement distinguée**, et la sonde le confirme (lue).

4. Scenario utilisateur

```
MESURES      cou 38,9 . taille 91,3 . case % laissee VIDE
CALCUL        _bfNavy -> 20.1 %
ENREGISTREMENT weightLog[0] = {kg:85.9, bf:20.1, bfSrc:'estime'}      [ok]
               toast : « Estimation enregistree : 20.1 % »           [ok]

RELECTURE     leanMassRecente() -> {lm:68.6, src:'pesee', nature:'SAISIE'} <<< ICI
MASSE MAIGRE   68.6 kg
BMR            Katch-McArdle -> 1852 (Mifflin aurait donne 1749)
CONTEXTE MILO  « cette masse maigre est une ESTIMATION, pas une mesure :
               elle est calculee a partir d un % de masse grasse
               qu il/elle a SAISI lui/elle-meme »                     <<< FAUX
```

[!!] La provenance devient fausse a state.js, entre l enregistrement et la relecture.

5. Erreur SEMANTIQUE - reelle et prouvee

Ce que Milo recoit, **mot pour mot**, mesure dans buildCoachContext :

Ce qui est stocke sur la ligne	La raison que Milo lit
bfSrc: 'estime'	[X] elle est calculée à partir d'un % de masse grasse qu'il/elle a SAISI lui/elle-même
bfSrc: 'mesure'	[ok] elle est calculée à partir d'un % de masse grasse qu'il/elle a SAISI lui/elle-même
bfSrc absent	[X] elle est calculée à partir d'un % de masse grasse qu'il/elle a SAISI lui/elle-même
bilan corporel (autre branche)	[ok] une balance mesure un poids et une impédance, puis ESTIME le reste avec la formule de son fabricant

La nuance a ne pas ecraser

Le bloc annonce **deja correctement** « *cette masse maigre est une ESTIMATION, pas une mesure* ». [*] Milo n est donc **pas** amene a croire a une mesure : il est amene a croire a la **mauvaise raison**. *C est un fait faux sur la personne - elle n a rien tape, l application a calcule a partir de ses centimetres - mais ce n est pas une mesure presentee comme telle.*

6. Effet NUMERIQUE reel - zero

Meme chiffre (20.1 %), meme profil synthetique, **seul bfSrc change** :

Cas	bfSrc	BMR	Methode	TDEE	Cible	nature annoncee
B estimation US Navy	estime	1852	katch	3321	3171	saisie
C meme %% tape a la main	mesure	1852	katch	3321	3171	saisie
D ancienne ligne	absent	1852	katch	3321	3171	saisie

[!!] B, C et D sont strictement indiscernables - en chiffres ET en texte. L etiquette ne change aucun calcul : l erreur est purement semantique.

Un fait numerique VOISIN, a ne pas confondre avec celui-ci

La **presence** de l estimation fait basculer Mifflin -> Katch-McArdle : **1749 -> 1852 kcal de BMR (+103), 3161 -> 3321 de TDEE (+160), 3011 -> 3171 de cible (+160)**. [*] [X] Mais cela **ne vient pas de la perte de provenance** : bmrDetail n a jamais exige une mesure, et ft-v1231 a explicitement decide qu une estimation pouvait etre enregistree. *C est une consequence de conception, pas le default etudie.*

7. Moteurs touches - seulement ce qui est prouve

Moteur	Touche par la PERTE DE PROVENANCE ?	Preuve
Texte envoye a Milo	[ok] OUI - une phrase fausse	sonde F, trois cas compares
Choix Mifflin / Katch	[X] non	bfSrc n entre pas dans bmrDetail
Valeur du BMR	[X] non	B = C = D = 1852
TDEE	[X] non	B = C = D = 3321
Cible calorique	[X] non	B = C = D = 3171

Moteur	Touche par la PERTE DE PROVENANCE ?	Preuve
Macros	[X] non	elles derivent de la cible, identique
Graphiques	[X] non	la courbe lit w.bf, pas nature
Affichage utilisateur	[X] non	le toast et la carte distinguent correctement

8. Rayon et severite - reclassement

Classement	Annonce	Apres mesure	Justification
Rayon	MULTI-MOTEUR	LOCAL	Une seule phrase, dans un seul bloc du contexte de Milo. Aucun calcul, aucun ecran, aucun export n en depend - refute par B = C = D.
Severite	MAJEUR	MOYEN	Ce qui l empeche de descendre a MINEUR : c est un fait faux sur la personne envoye a Milo, qui peut le lui repeter. Ce qui l empeche de rester MAJEUR : aucune decision, aucun chiffre, aucun conseil n en depend , et le bloc annonce deja correctement « estimation ».

9. Le cas bfSrc absent - intervient-il ici ?

Non, pas differemment. leanMassRecente ne lit **jamais** bfSrc : son absence est donc **sans effet sur ce chemin**, et le cas D rend exactement B et C. [*] Le soupcon du rapport precedent - « *l absence est interpretee differemment selon les endroits* » - reste ouvert **ailleurs** : bfDerniere et _bfRemplacableParEstime, eux, le lisent. **POINT CONNEXE A VERIFIER PLUS TARD.**

10. Points connexes - notes, PAS suivis

Point connexe a verifier plus tard	Ancre
tracking.js ecrit bfSrc = BF_MESURE pour un bilan corporel , alors qu une balance a impendance estime le %% (R32, ecrite dans ce depot). Meme famille, autre porte.	tracking.js:1990
src: 'pese' est annonce a Milo pour une valeur qui vient d un ruban a mesurer , pas d une balance.	state.js (leanMassRecente)
L absence de bfSrc est lue differemment par bfDerniere et _bfRemplacableParEstime.	tracking.js:840 et :845

11. Correction envisageable PLUS TARD - rien n est applique

```
// state.js - aujourd'hui
cand.push({ ..., src:'pese', nature:'saisie' });

// ce qu'elle POURRAIT devenir
cand.push({ ..., src:'pese', nature: w.bfSrc === 'estime' ? 'estimee_navy'
                                     : w.bfSrc === 'mesure' ? 'saisie'
                                     : 'inconnue' });
```

Avec la branche correspondante dans coach.js : le ternaire y porte **2 tests** de nature, il en faudrait deux de plus.

La borne a ne pas franchir

[X] **Ne pas inventer la provenance d une ligne ancienne.** 'inconnue' doit rester un **troisieme etat**, pas retomber sur 'saisie'. [*] C est mot pour mot la **regle d or #16**, et c est le piege que ft-v1231 et ft-v1227 ont deja paye : *on fabriquerait la provenance avec le mecanisme construit pour ne pas l inventer.*

Etat du depot, verifie par git a la generation

AUCUN FICHIER SERVI N EST MODIFIE (state.js, tracking.js, coach.js, sw.js). Aucun bump - le cache sert toujours **ft-v1234**. Aucune publication. [*] La sonde tourne sur un profil **SYNTHETIQUE** declare : aucune valeur de ce dossier ne peut etre attribuee a une personne reelle.

Genere par tools/gen_provenance_bf_pdf.py. Ses **39 gardes** relisent chaque ancre dans le code servi et recomptent chaque chiffre depuis la sonde. [!!] Le garde central extrait leanMassRecente de state.js et **refuse de produire si elle lit bfSrc** - c est-a-dire si le default decrit etait corrige.